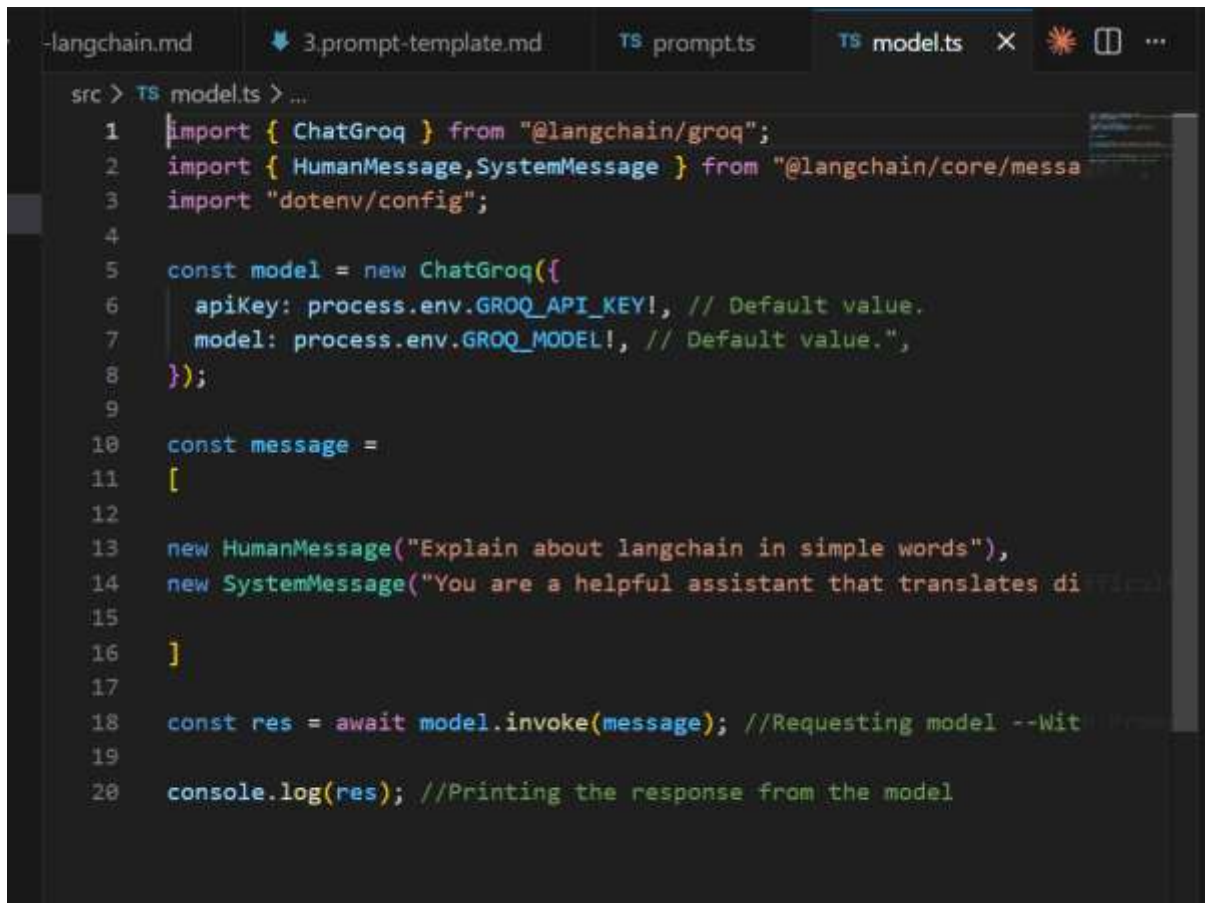


Introduction to Langchain:

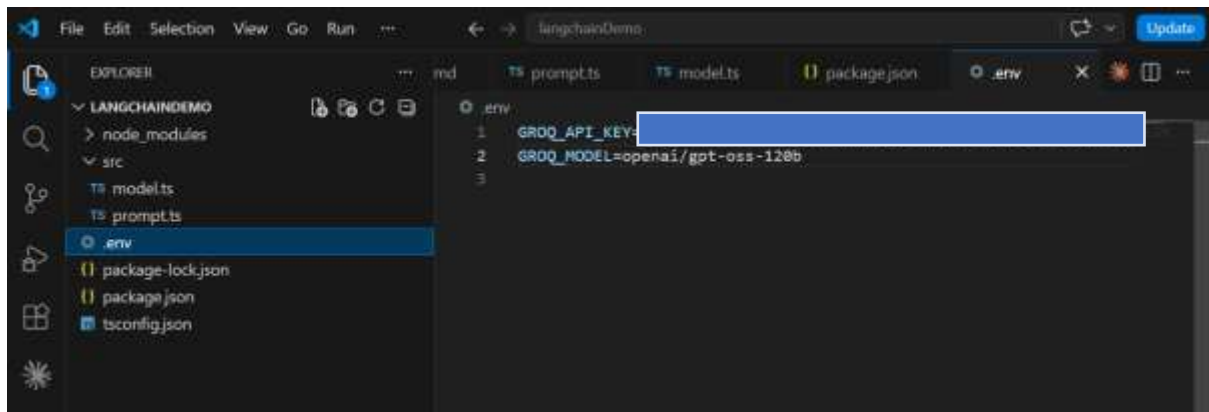
Model.ts

Added Model.ts file under src folder and added the implementation

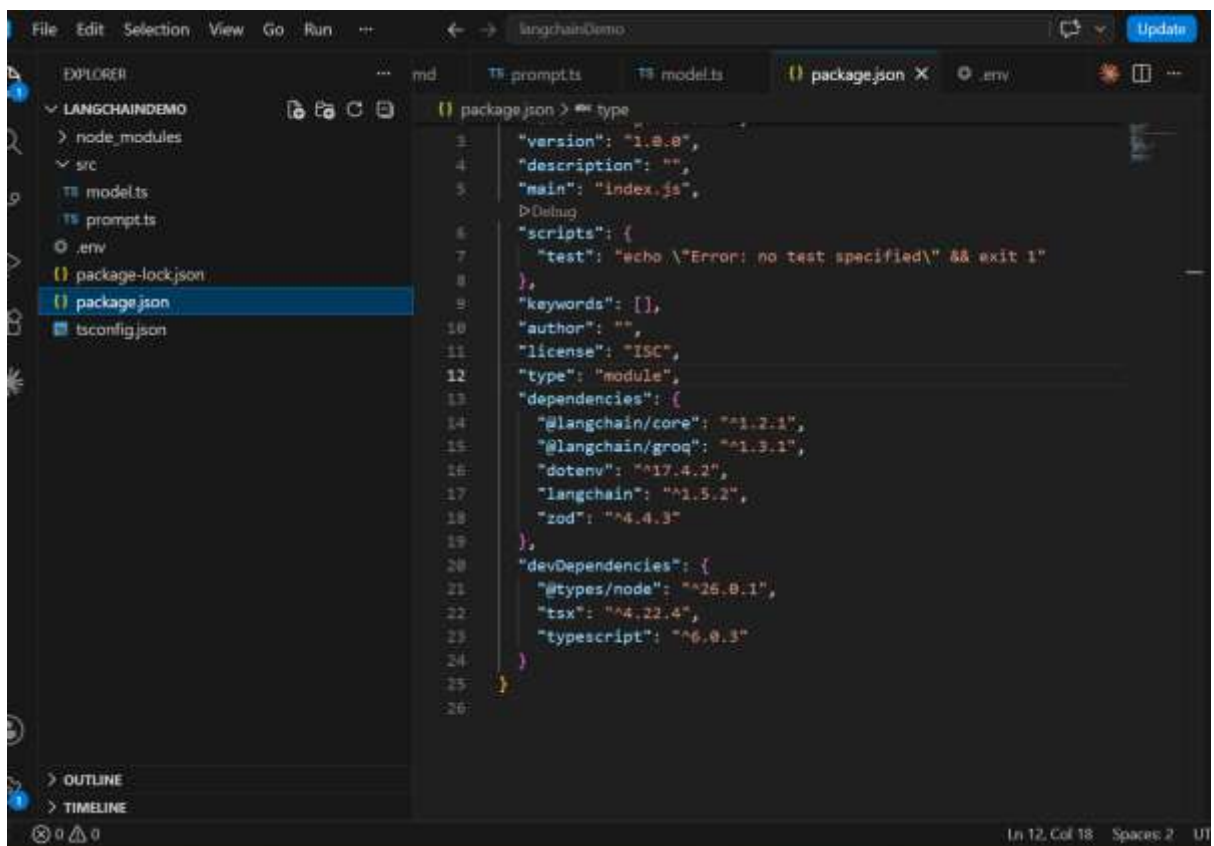


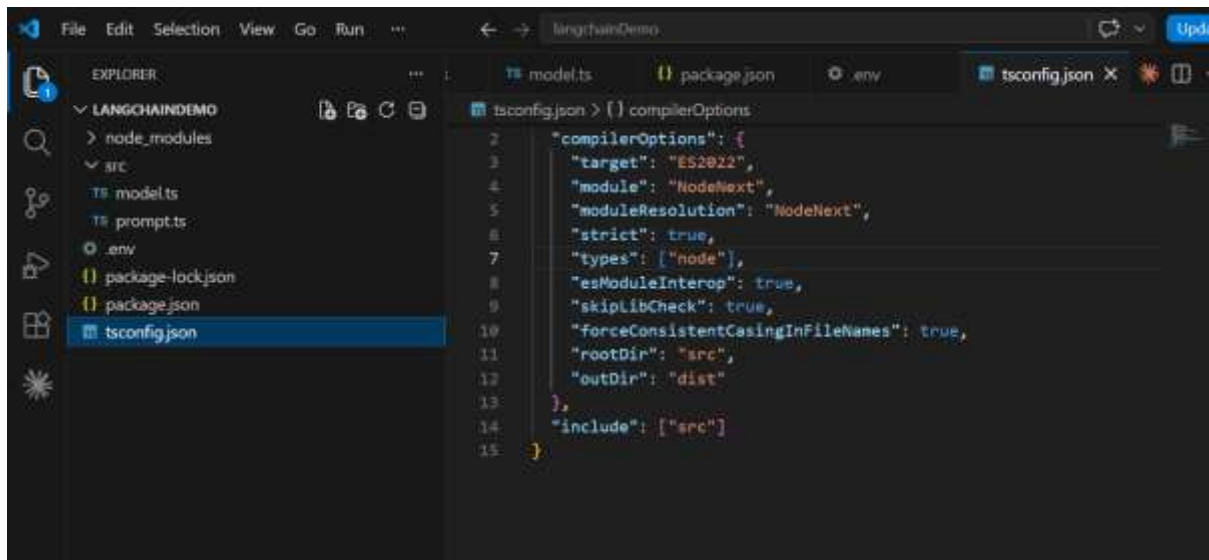
```
src > TS model.ts > ...
1  import { ChatGroq } from "@langchain/groq";
2  import { HumanMessage, SystemMessage } from "@langchain/core/messages";
3  import "dotenv/config";
4
5  const model = new ChatGroq({
6    apiKey: process.env.GROQ_API_KEY!, // Default value.
7    model: process.env.GROQ_MODEL!, // Default value.",
8  });
9
10 const message =
11 [
12
13   new HumanMessage("Explain about langchain in simple words"),
14   new SystemMessage("You are a helpful assistant that translates difficult words into simple words"),
15 ]
16 ]
17
18 const res = await model.invoke(message); //Requesting model --With Prompt
19
20 console.log(res); //Printing the response from the model
```

Updated Groq APIKey and Model in the env file:

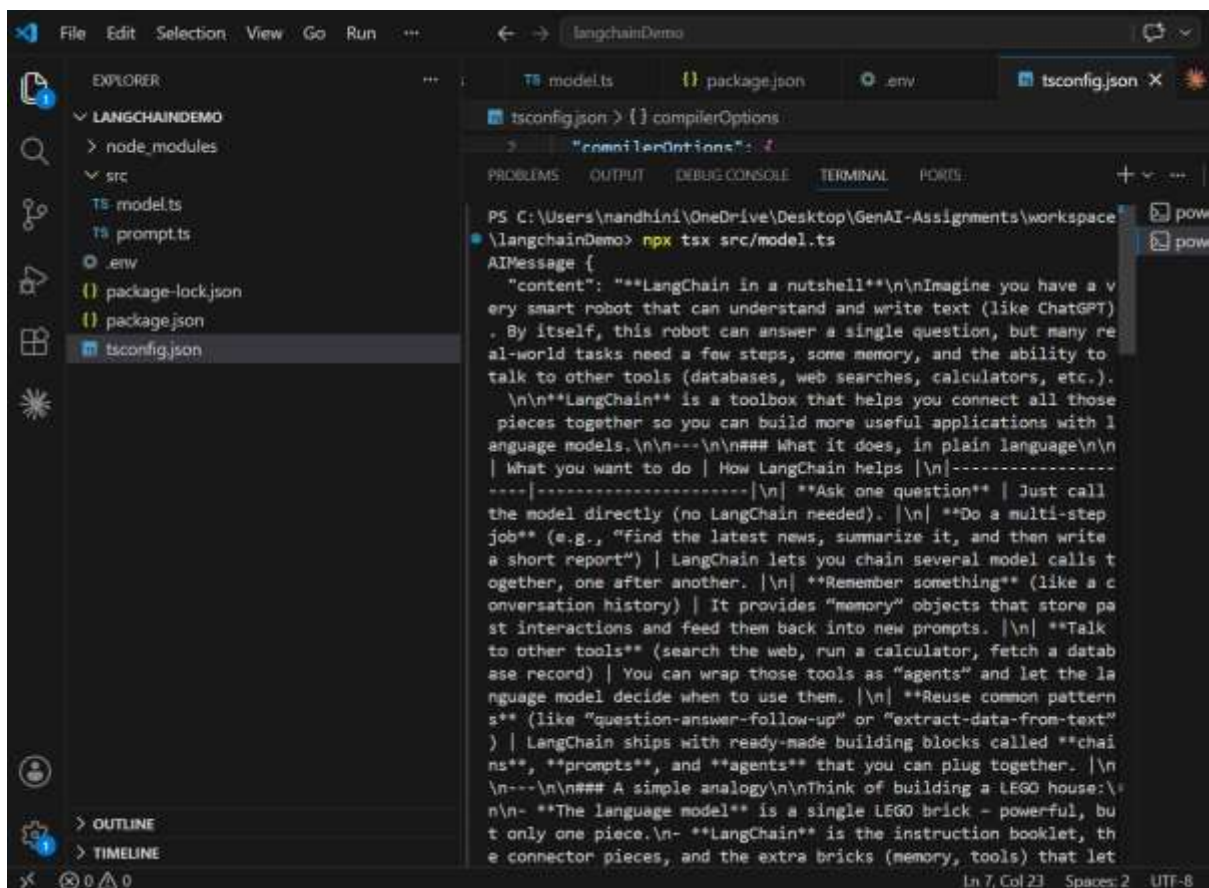


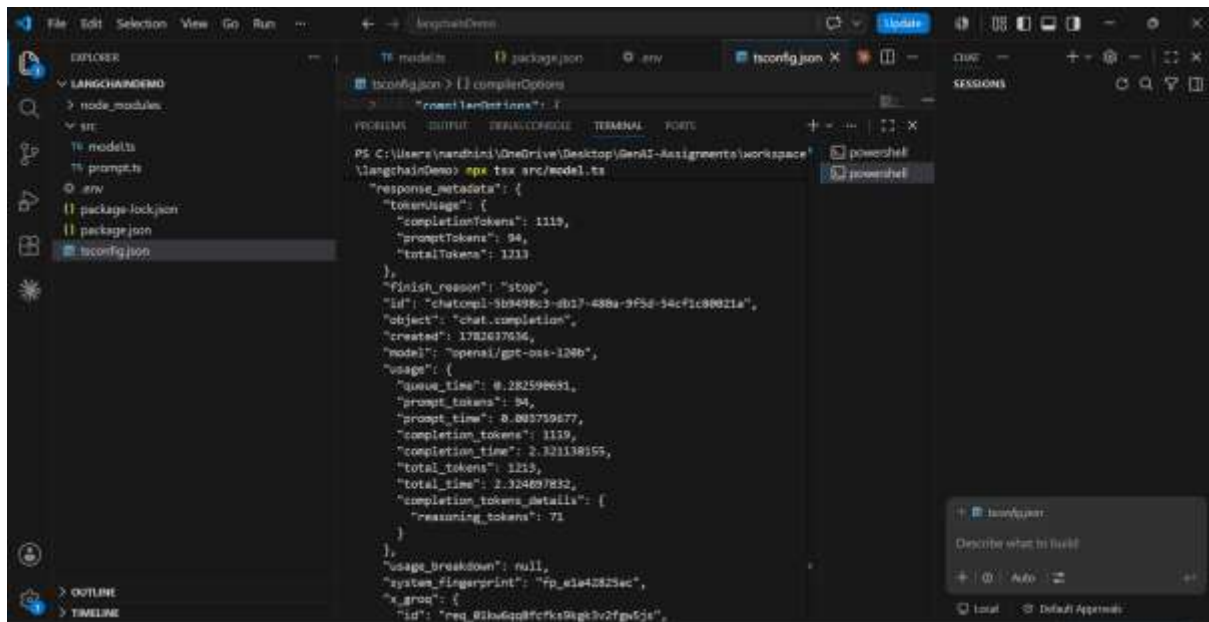
Updated package.json and tsconfig.json:





Running the model.ts file in the terminal:

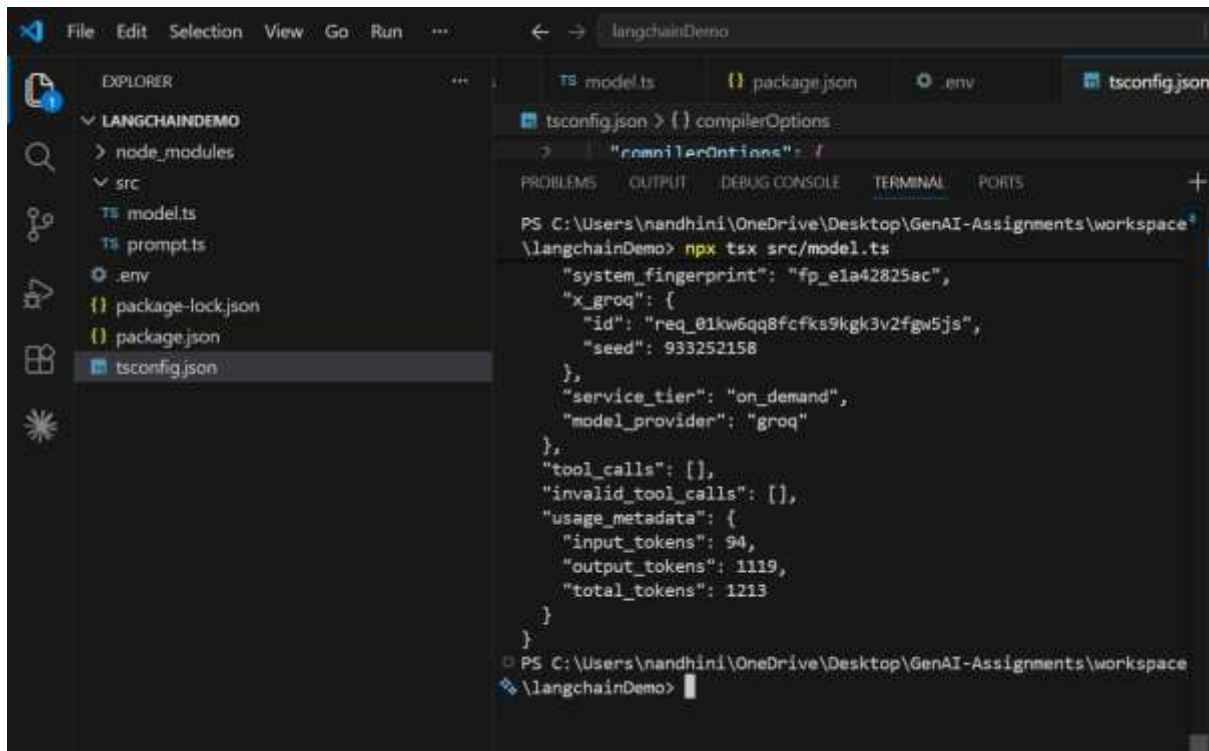




```
tsconfig.json > {} compilerOptions
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  }
}

PS C:\Users\nandhini\OneDrive\Desktop\GenAI-Assignments\workspace\langchainDemo> npx tsx src/model.ts
{"response_metadata": {"tokens": {"completion_tokens": 1119, "prompt_tokens": 94, "total_tokens": 1213}, "finish_reason": "stop", "id": "chatcmpl-5b9498c3-d817-488a-9f5d-54cf1c98021a", "object": "chat.completion", "created": 1782637636, "model": "openai/gpt-oss-120b", "usage": {"queue_time": 0.282598631, "prompt_tokens": 94, "prompt_time": 0.003759677, "completion_tokens": 1119, "completion_time": 2.321138155, "total_tokens": 1213, "total_time": 2.324897832, "completion_tokens_details": {"reasoning_tokens": 71}}, "usage_breakdown": null, "system_fingerprint": "fp_e1a42825ac", "x_groq": {"id": "req_01kw6qq8fcfs9kgk3v2fgw5js", "seed": 933252158}}, {"system_fingerprint": "fp_e1a42825ac", "x_groq": {"id": "req_01kw6qq8fcfs9kgk3v2fgw5js", "seed": 933252158}}, {"service_tier": "on_demand", "model_provider": "groq"}], "invalid_tool_calls": [], "usage_metadata": {"input_tokens": 94, "output_tokens": 1119, "total_tokens": 1213}}
}

PS C:\Users\nandhini\OneDrive\Desktop\GenAI-Assignments\workspace\langchainDemo>
```



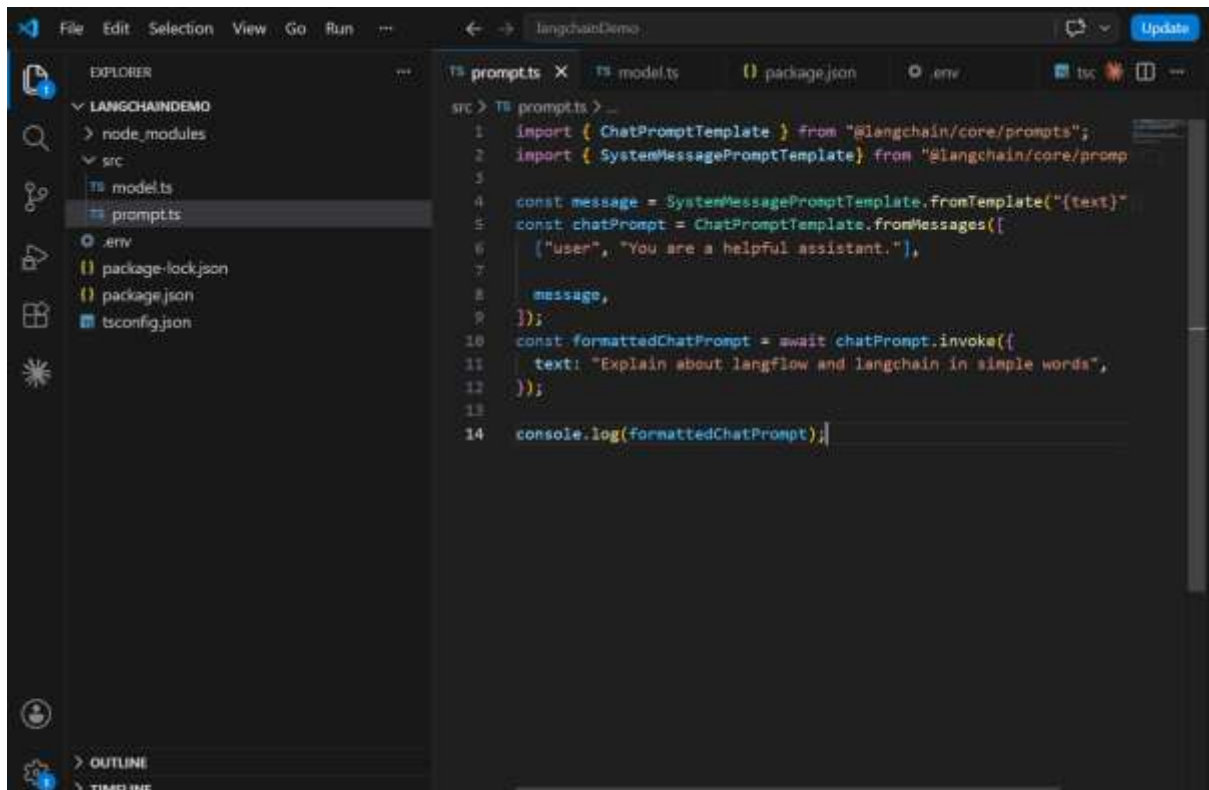
```
tsconfig.json > {} compilerOptions
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  }
}

PS C:\Users\nandhini\OneDrive\Desktop\GenAI-Assignments\workspace\langchainDemo> npx tsx src/model.ts
{"system_fingerprint": "fp_e1a42825ac", "x_groq": {"id": "req_01kw6qq8fcfs9kgk3v2fgw5js", "seed": 933252158}}, {"system_fingerprint": "fp_e1a42825ac", "x_groq": {"id": "req_01kw6qq8fcfs9kgk3v2fgw5js", "seed": 933252158}}, {"service_tier": "on_demand", "model_provider": "groq"}], "invalid_tool_calls": [], "usage_metadata": {"input_tokens": 94, "output_tokens": 1119, "total_tokens": 1213}}
}

PS C:\Users\nandhini\OneDrive\Desktop\GenAI-Assignments\workspace\langchainDemo>
```

Prompt.ts:

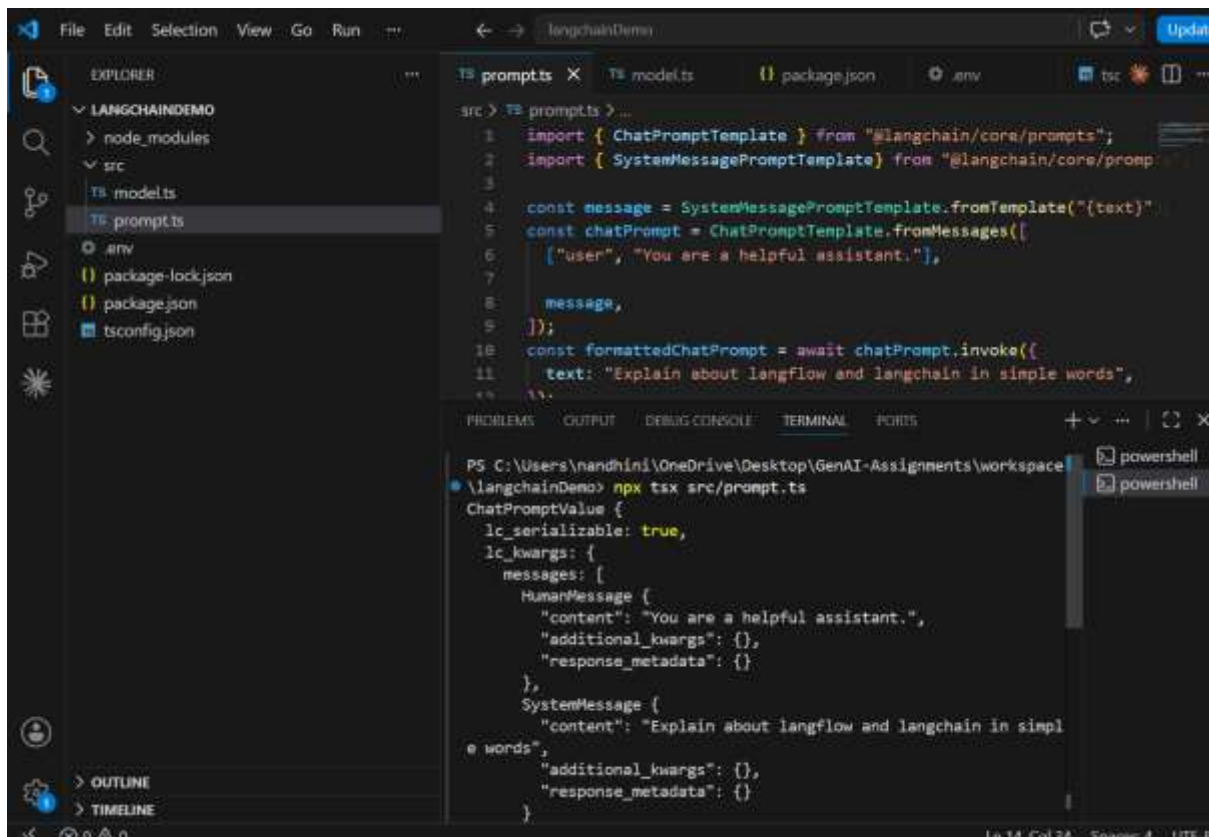
Added prompt.ts file and updated the code:



The screenshot shows the Visual Studio Code editor interface. The Explorer sidebar on the left displays the project structure for 'LANGCHAINDEMO', including 'node_modules', 'src', 'models.ts', 'prompt.ts', '.env', 'package-lock.json', 'package.json', and 'tsconfig.json'. The 'prompt.ts' file is selected and its content is displayed in the main editor window. The code imports 'ChatPromptTemplate' and 'SystemMessagePromptTemplate' from '@langchain/core/prompts'. It defines a 'message' using 'SystemMessagePromptTemplate.fromTemplate' and a 'chatPrompt' using 'ChatPromptTemplate.fromMessages'. The 'chatPrompt' is then invoked with a text prompt, and the result is logged to the console.

```
src > TS prompt.ts > ...
1 import { ChatPromptTemplate } from "@langchain/core/prompts";
2 import { SystemMessagePromptTemplate } from "@langchain/core/prompts";
3
4 const message = SystemMessagePromptTemplate.fromTemplate("{text}");
5 const chatPrompt = ChatPromptTemplate.fromMessages([
6   ["user", "You are a helpful assistant."],
7   message,
8 ]);
9
10 const formattedChatPrompt = await chatPrompt.invoke({
11   text: "Explain about langflow and langchain in simple words",
12 });
13
14 console.log(formattedChatPrompt);
```

Executed `npx tsx src/prompt.ts` command in the terminal and got the response successfully:

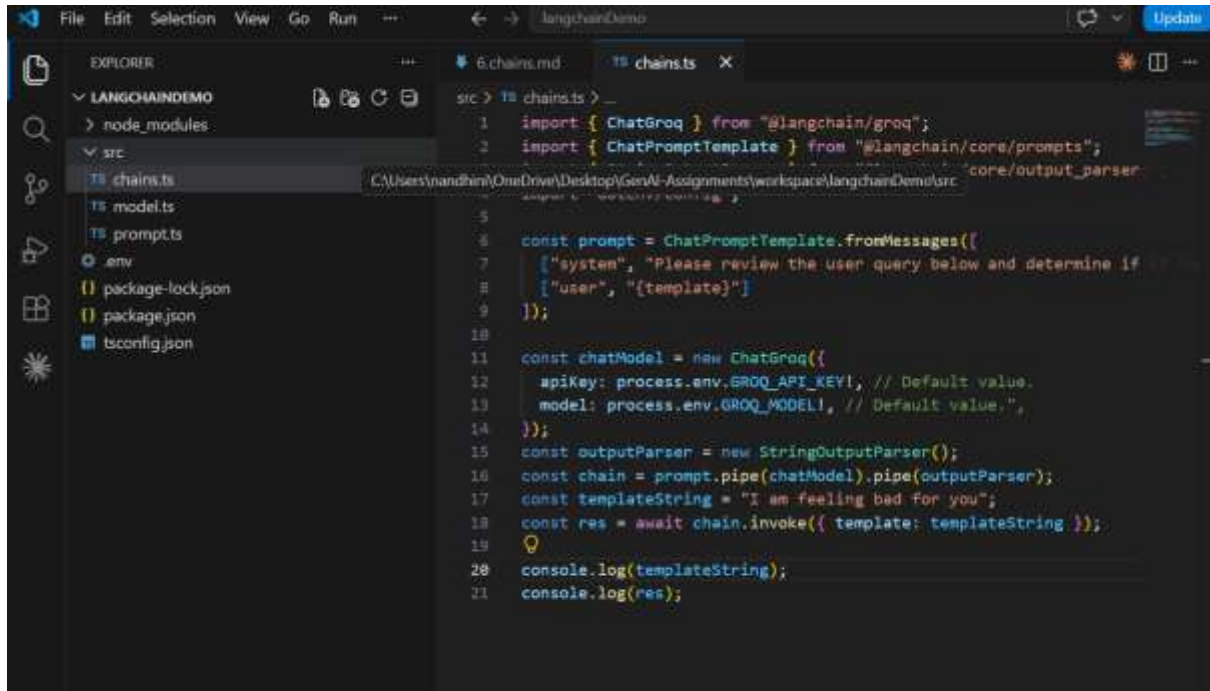


The screenshot shows the Visual Studio Code editor interface with the 'prompt.ts' file selected. The terminal window at the bottom displays the output of the command `npx tsx src/prompt.ts`. The output is a JSON object representing the 'ChatPromptValue' with its serializable status, kwargs, and a list of messages (HumanMessage and SystemMessage).

```
PS C:\Users\nandhini\OneDrive\Desktop\GenAI-Assignments\workspace>
.\langchainDemo> npx tsx src/prompt.ts
ChatPromptValue {
  lc_serializable: true,
  lc_kwargs: {
    messages: [
      HumanMessage {
        content: "You are a helpful assistant.",
        additional_kwargs: {},
        response_metadata: {}
      },
      SystemMessage {
        content: "Explain about langflow and langchain in simple words",
        additional_kwargs: {},
        response_metadata: {}
      }
    ]
  }
}
```

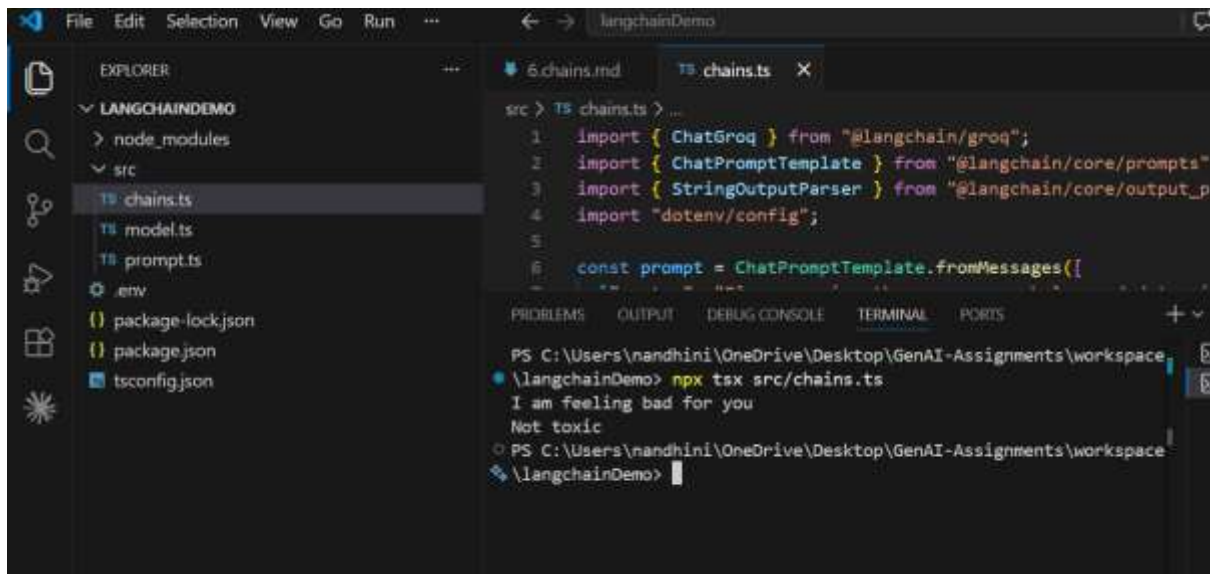

Chains.ts

Added chains.ts file and updated the code:



```
1 import { ChatGrok } from "@langchain/grok";
2 import { ChatPromptTemplate } from "@langchain/core/prompts";
3 import { StringOutputParser } from "@langchain/core/output_parsers";
4 import "dotenv/config";
5
6 const prompt = ChatPromptTemplate.fromMessages([
7   ["system", "Please review the user query below and determine if it is toxic."],
8   ["user", "{template}"]
9 ]);
10
11 const chatModel = new ChatGrok({
12   apiKey: process.env.GROQ_API_KEY!, // Default value.
13   model: process.env.GROQ_MODEL!, // Default value.
14 });
15 const outputParser = new StringOutputParser();
16 const chain = prompt.pipe(chatModel).pipe(outputParser);
17 const templateString = "I am feeling bad for you";
18 const res = await chain.invoke({ template: templateString });
19
20 console.log(templateString);
21 console.log(res);
```

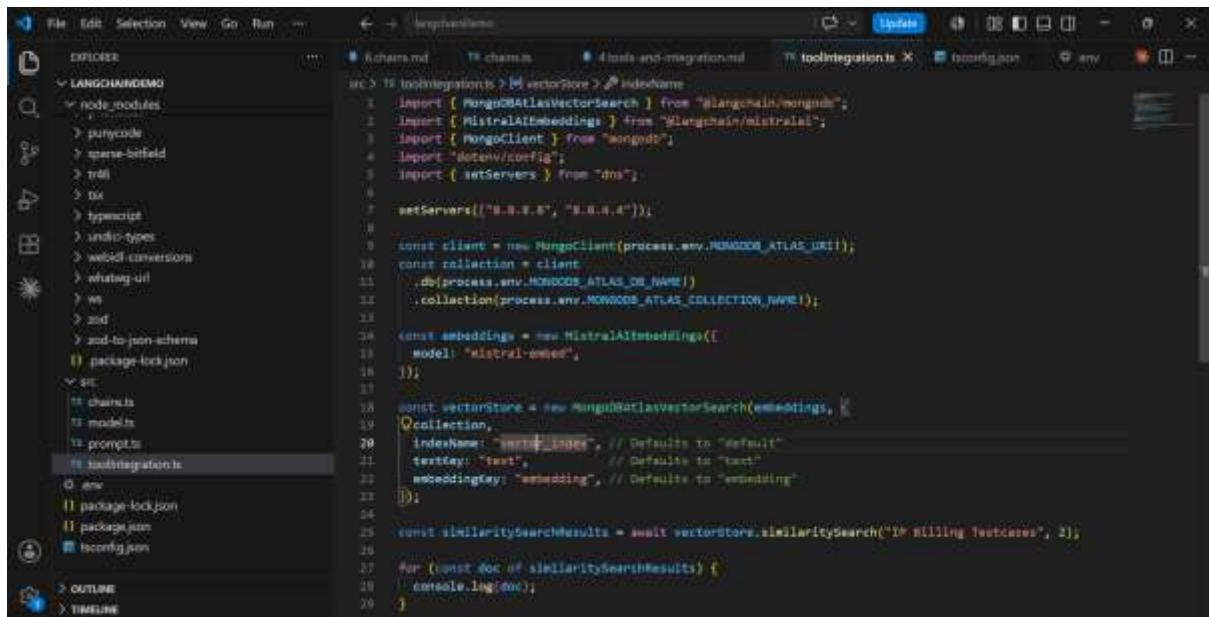
Executing the chains.ts file, printing both templateString given and the LLM result on analysis:



```
PS C:\Users\nandhini\OneDrive\Desktop\GenAI-Assignments\workspace\langchainDemo> npx tsx src/chains.ts
I am feeling bad for you
Not toxic
PS C:\Users\nandhini\OneDrive\Desktop\GenAI-Assignments\workspace\langchainDemo>
```

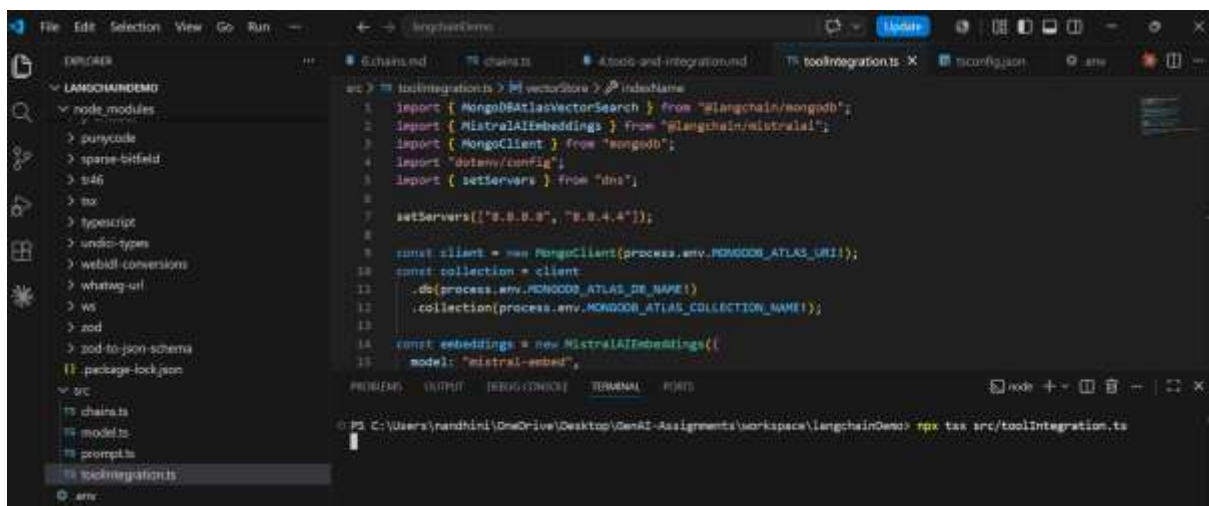
toolIntegration.ts

Added toolIntegration.ts file with the code



```
1 import { MongoClient } from "mongodb";
2 import { MistralAIEmbeddings } from "langchain/mistralai";
3 import { MongoAtlasVectorSearch } from "langchain/mongodb";
4 import { dotenv/config } from "dotenv/config";
5 import { setServers } from "dns";
6
7 setServers(["8.8.8.8", "8.8.4.4"]);
8
9 const client = new MongoClient(process.env.MONGODB_ATLAS_URI);
10 const collection = client
11   .db(process.env.MONGODB_ATLAS_DB_NAME)
12   .collection(process.env.MONGODB_ATLAS_COLLECTION_NAME);
13
14 const embeddings = new MistralAIEmbeddings({
15   model: "mistral-embed",
16 });
17
18 const vectorStore = new MongoAtlasVectorSearch(embeddings, {
19   collection,
20   indexName: "vector_index", // Defaults to "default"
21   textKey: "text", // Defaults to "text"
22   embeddingKey: "embedding", // Defaults to "embedding"
23 });
24
25 const similaritySearchResults = await vectorStore.similaritySearch("I'm killing testcases", 2);
26
27 for (const doc of similaritySearchResults) {
28   console.log(doc);
29 }
```

File is not executing due to loop issue, Informed Prabhakaran and Sudharshan, will try again when the issue is resolved:



```
1 import { MongoClient } from "mongodb";
2 import { MistralAIEmbeddings } from "langchain/mistralai";
3 import { MongoAtlasVectorSearch } from "langchain/mongodb";
4 import { dotenv/config } from "dotenv/config";
5 import { setServers } from "dns";
6
7 setServers(["8.8.8.8", "8.8.4.4"]);
8
9 const client = new MongoClient(process.env.MONGODB_ATLAS_URI);
10 const collection = client
11   .db(process.env.MONGODB_ATLAS_DB_NAME)
12   .collection(process.env.MONGODB_ATLAS_COLLECTION_NAME);
13
14 const embeddings = new MistralAIEmbeddings({
15   model: "mistral-embed",
16 });
17
18 const vectorStore = new MongoAtlasVectorSearch(embeddings, {
19   collection,
20   indexName: "vector_index", // Defaults to "default"
21   textKey: "text", // Defaults to "text"
22   embeddingKey: "embedding", // Defaults to "embedding"
23 });
24
25 const similaritySearchResults = await vectorStore.similaritySearch("I'm killing testcases", 2);
26
27 for (const doc of similaritySearchResults) {
28   console.log(doc);
29 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL RUNS

PS C:\Users\nandhini\OneDrive\Desktop\GenAI-Assignments\workspace\langchainDemo> npm run ts:dev